The EPFL Whisperer: your personal TA powered by natural language processing

nlPAP team: Paul Boulenger, Alexia Dormann, Pierre Hellich
EPFL Lausanne, Switzerland

***Abstract*—In the past few years Language models (LM) have become more and more performant. They are now capable of generating consistent and robust reasoning on a wide variety of tasks. One of the exciting applications of these models is education. In this paper, we introduce the EPFL Whisperer, an AI-powered tool designed to provide personalized guidance and answer EPFL course-related questions.**

A two-step training approach is presented. First, we fine-tuned a pre-trained GPT2 model using supervised learning using distil data from ChatGPT. Next, we present the reinforcement learning from human feedback (RLHF) framework to further refine our model and align it with human preferences. This is done using a reward model trained on human-ranked pairs of demonstrations to predict human preference and the Proximal Policy Optimization (PPO) algorithm. We demonstrate the effectiveness of the supervised fine-tuning and RLHF techniques in improving the model's performance and alignment with user intent.

Through extensive experimentation, we highlight the challenges of training and fine-tuning language models for educational purposes. The EPFL Whisperer represents a proof of concept towards personalized education and showcases the potential of AI-powered tools. Indeed, even though the performance achieved here is not yet satisfactory, we are sure that given more computing budget and time our tutor could become a powerful tool to empower EPFL students in their studies.

I. Introduction

"How do you integrate this expression? What's Gauss' law again? How is Prof. Bosselut such an amazing teacher? How does polymorphism work? " Antoine has many questions. He has been raising his hand for 30 minutes now, but no TA in sight... Ah, it would be nice if he had a personal tutor! Indeed, students at EPFL need to learn and master a variety of topics in a limited time. We present here an AI-powered tool, the EPFL-Whisperer. Designed to provide personal guidance to any EPFL students and answer all their course-related questions. So, that they can finally understand what group isomorphism is!

The emergence of Large Language Models (LLMs) has the potential to revolutionize education. Making it more personalized and accessible to all. Indeed, models such as ChatGPT are able to be useful assistants when prompted with questions from a multitude of topics (Ling et al. [1]). Even though OpenAI's ChatBot was released less than a year ago it has become a regular study companion for many students. However, one quickly realizes that the tool is often inaccurate and has difficulty producing coherent reasoning in more technical domains.

For example, when queried with basic math and logic problems, it performs relatively well. But it struggles to demonstrate a deep understanding of the reasoning behind more complex problems (Pleveris et al. [2]). Similarly, its efficacy as a programming assistant is limited. (Tian et al. [3]). More generally, even though LLMs display impressive performance on a range of benchmarks, they still often do not follow user instructions. They need to be aligned to the user intent (Ouyang et al. [4]).

Supervised fine-tuning of GPT2: A solution, besides increasing the model and dataset sizes, is the training of specialized models. We describe here the training of such a specialized Chatbot for question answering for EPFL courses. First, using ChatGPT, we collected demonstrations of the intended behaviour for an EPFL question set. In particular, we used the interactions collected in the context of the CS-552 course and additional demonstrations collected by prompting ChatGPT with the question and the expected answer as a hint. In addition, we used a public dataset of a dump of Stack-Exchange, a popular Q&As forum [5]. Only the questions relevant to EPFL students were retained. Due to computational and time constraints, we were only able to fine-tune a relatively small-scaled language model, GPT2.

First off, the pre-trained GPT2 was fine-tuned using the next token prediction framework on the stack exchange dataset. (Radford et al. [6]) As the dataset contains demonstrations of questions answered in science-related topics, it was used to first specialize the model. Then, the model was further fine-tuned using the distilled ChatGPT interactions.

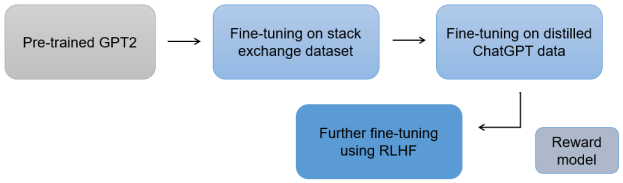

Fig. 1. General training workflow of our AI tutor. a) Supervised fine-tuning using good demonstrations b) Further fine-tuning using the RLFH framework using a custom reward model

Further fine-tuning using reinforcement learning from human feedback (RLHF): Next, to further fine-tuned our model and align it to human preference, we explored the RLFH framework (Christiano et al. [7], Ouyang et al. [4]). As its name suggests, this method uses the RL paradigm. To train the reward model, used as the reward function, we gathered human-ranked pairs of demonstrations for the task of interest. As the generations of such demonstrations by humans would be time-consuming and expensive, we distilled the data from

ChatGPT. Using this paired data and starting from the pre-trained DistilGPT2, we trained the model to predict human preference. Finally, we highlight the usefulness of the PPO algorithm to align the model to user intent.

II. Related work

Training a dialogue model and alignment to human preference (RLHF) In recent years, LLMs such as ChatGTP, have reached impressive performances. Indeed, since its deployment, many users have adopted it and used it as an assistant for work, education, or entertainment. However, even though the training procedure of its sibling model, *InstructGPT* was recently published (Ouyang et al. [4]), the exact training recipe and architecture remained unknown. Moreover, it still often outputs false information, exhibits toxic behaviour, and perpetuates biases such as misogyny, racism, etc,... More research is needed to make such models open-sourced and mitigate these unintended behaviours.

Initiatives from academia attempt to address these issues. For instance, Standford's Alpaca [8], Vicuna [9], and Berkeley's Koala [10] have recently made available their training procedure. Interestingly, they claimed to have similar performances as OpenAI's model even with far fewer parameters than GPT3, the base model used to train ChatGPT. All approaches started from LLaMA, 7 or 13 B parameters foundation LM. (Touvron et al. [11])

All these models use a similar training framework. First, data is distilled from LLMs. For instance, the researchers behind the Alpaca model used text-DaVinci-003 to collect instructions for the supervised training of their model. The others used public user-shared ChatGPT interactions for training. Indeed, collecting human-written demonstrations is time-consuming and expensive. Moreover, it has been shown that using strong LLMs to generate the instructions increased the performance of the trained model compared to public instructions datasets (Wang et al. [12]). The base model is fine-tuned using the collected data using the next token prediction objective. In addition, *InstructGPT* introduces the concept of RLHF for fine-tuning a dialogue model (Ouyang et al. [4])

Evaluation of dialogue models Interestingly, human evaluation was used to access the performance. The testers were provided with a set of interactions with the models and were asked to compare and rate them. However, this approach suffers major drawbacks. Indeed, when models reach SOTA, it becomes difficult for humans to compare them. Moreover, it is a time-consuming and heavily biased task. The testers may have different ideas of what is a good interaction. To mitigate some of the issues, the authors of Vicuna introduced an automatic evaluation using GPT4. GPT4 outputs a score and the reasoning behind this score for each demonstration. [9]

Limitations of the approach As noted by the authors, all these models still suffer from common shortcomings of LMs. Indeed, they display toxic behaviour, hallucination, and biases. Furthermore, the training procedure and architecture of ChatGPT, one of the best-performing models, still remained unknown. Additionally, even though the models presented here are much smaller than GPT3 for instance, they are still relatively big, with a few billion parameters. Their training still requires a relatively large amount of compute. Lastly, even if they achieve impressive performance on a multitude of tasks, they are still lagging behind on more technical tasks such as math and coding tasks.

Here we address some of these issues. Truly, our approach is heavily inspired by the one outlined in the *IntructGPT* paper [4] and the aforementioned Alpaca, Koala, and Vicuna. [8], [10], [9]. We reproduced the training procedure of fine-tuning using distilled ChatGPT data and explored the RLHF framework. Nevertheless, our base model is much smaller with 355M parameters and aimed to be specialized in science-related questions.

III. Methods and experimental details

In this section, we provide a more detailed description of our methodology. As previously mentioned, our work is heavily inspired by previous work. The general workflow is as follows (Fig. 1): We start from a pre-trained model and fine-tuned it on demonstrations of the intended behaviour to align it using supervised learning. Then, we investigated further fine-tuning using RLHF. To this effect, a reward model was trained to predict human preference.

A. Data collection

EPFL being an engineering school, students will have science- and coding-related questions. We were provided with questions from exercise sheets and exams from various courses taught at EPFL. As we want our model to mimic the behaviour of a human TA, TA-like explanations were gathered (Fig. 2). The quality of the fine-tuned model is heavily dependent on the quality of the data. Ideally, one would gather such data written by human experts. However, since we did not have access to such experts, we opted to distil our data from ChatGPT.

Knowledge Distillation from ChatGPT When prompted correctly, OpenAi's ChatBot is able to generate demonstrations of surprising quality. Still, to ensure the quality of the data, each interaction with the API was rated by labellers (Students from the CS552 course) in terms of their confidence in the output.

However, as students were tasked to define their custom prompting strategy, we do not go into details of the different strategies. In general, the following strategies elicited good outputs from the model:

- *General prompting guidelines*: Using positive language (i.e. no negation) to instruct the model encouraged it to generate more relevant reasoning. Additionally, using clear and specific instructions is key for the quality of the demonstrations. Defining the role of the model as a Tutor in the specific domain of the question leads to better results.
- *Using English prompts*: Indeed, some of the questions are in French. But, ChatGPT's French outputs are of lesser

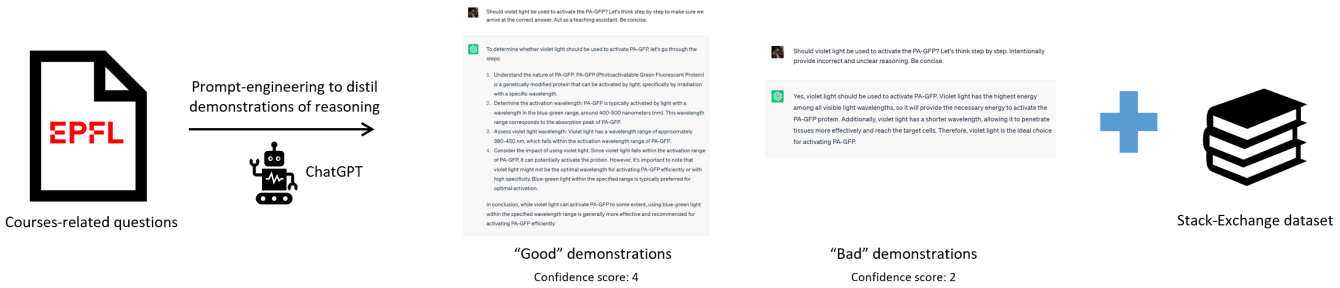

Fig. 2. Diagram of the data collection using ChatGPT API and additional Stack Exchange dataset

quality. Therefore, prompting the model in English is preferred.

- *Zero-shot-CoT*: Due to our lack of time and expertise, Chain-of-Thought prompting was not feasible (Wei et al.[13]). Therefore, we decided to use Zero shot-Cot, a method proposed by Kojima et al.(2023) [14]. For instance, adding "Let's think step by step" after the question leads to more accurate reasoning.

In addition to the student-gathered data, we also collected more data using the ChatGPT API. In particular, we used the provided solution to the questions using the following prompts respectively for open-ended and multiple choices questions to guide the model to generate good demonstrations. A method that has been recently shown to improve performance (Li et al. [15]):

- *<Question>: Let's think step by step to make sure we arrive at the following answer: <Answer>*
- *<Question><Choices>: Let's think step by step to make sure we arrive at the following answer: <Answer>*

with the following system instructions: *Act as a teaching assistant. Be concise*

As we will detail in a subsequent section to train a reward model, we will also need "bad" demonstrations to build comparisons of interactions. As only part of the generated data was inspected we did not build pairs with the student-gathered data. Indeed, we were unsure how they compared. Instead, we ask ChatGPT to output intentionally bad reasonings and explanations. This was done using the following prompt (same for questions with multiple choices):

- *<Question>Create incorrect, inconsistent, and unclear reasoning to get the following answer: <Answer>*

with the system level instruction: *Act as a bad teaching assistant. Be concise. Explain badly your reasoning.*

To make sure of the quality of the data, a couple of dozen interactions were inspected for both good and bad demonstrations. The outputs were of very satisfying quality for the good and bad demonstrations. All good demonstrations were attributed a confidence score of 5 and 1 for the bad ones.

Additional science-related dataset Lastly, we also used a public dataset of a dump of Stack-exchange available on Hugging Face [5]. Only questions relevant to EPFL students were kept. This included:

- Bioinformatics, Chemistry, Data science, Earth science, Mathematics

Per their term of service, content posted on Stack Exchange is under cc-by-sa 4.0 licensing. Meaning that the data can be used given the proper attribution and a link to the licence: *https://creativecommons.org/licenses/by-sa/4.0/*. The data is already anonymized, therefore, we do not take any additional steps to ensure the privacy of the users. However, it is important to note that is possible that some private data remains in the data. Another point to consider is the intellectual property. Indeed, LLMs by design reproduce work found on the web, which has many legal implications. The data must be under permissive licensing that allows the use and modifications of the data. This is the case here. Next, one should consider the biases that might be present in such data. Indeed, most people creating content on forums are often white, native English-speaker, and male. Therefore, the type of questions and the perspective represented in the dataset may be biased.

Collection of additional questions for RLHF Finally, we needed additional prompts for the fine-tuning of our model using RLHF. Indeed, reusing the same prompts may lead to biased training and reduce its ability to learn from new interactions. Our aim was to obtain additional questions related to the same topics covered in the original question set. Thus, we requested ChatGPT to provide one follow-up question for each question. This is done using the following prompt:

- *Can you come up with one follow-up question concerning the following question? : <Question>*

with the system-level instruction: *Act as a student. Be concise. Answer in English.*

B. Data pre-processing

The data was pre-processed slightly differently for the supervised training of the tutor and the training of the reward model. We highlight the common steps:

- *Translation of French interactions:* We translated all French interactions. To determine if the text was in French we used the *langdetect* python library. For all texts detected as French, we ask ChatGPT to translate them into English with the following prompt:
  - *Precisely translate the following text to English. <Text >*

The quality of the translation was first accessed on a couple of dozen examples, which was deemed satisfactory. We translated all French interactions as the base model, GPT2, was only trained on English text. Moreover, due to the relatively small capacity of the model (352M parameters) and the lack of French data. We argue that the model would have difficulty learning to answer in French as well. Therefore, we decided to focus on its performance in English. Besides, we also investigated multilingual LMs, however, all of them had a relatively small maximum tokens length for the input.

- *Truncation of inputs to maximum length tokens:* Most LMs have a relatively limited input length, most often 512. However, the interactions are typically quite long, with a median of about 2100 tokens. As a compromise between input length and compute budget, we opted for GPT2 and DistilGPT2. They can take inputs up to 1024 tokens. We also explored larger models that could allow for very long sequences. However, the fine-tuning of such a model, for example, Longformer, comes with a high computing cost [16].
  Most of the demonstrations are still longer than 1024 tokens. A simple heuristic was applied to truncate the inputs. If the interaction was longer than 1024 tokens, we only kept the last user and assistant interactions as they were found the most relevant. If they were still too long, they were truncated to the appropriate lengths. Notably, for the supervised training of the tutor model, if the answer was too long, it was divided into equal parts.

Additionally, to ensure the quality of the data all the demonstrations with low scores were filtered out for the supervised training of the tutor model. We defined a threshold at a confidence score of 4.

As aforementioned, for the training of the reward model, pairs of demonstrations are needed. Indeed, scores are often noisy and may heavily vary depending on the labeller [17]. Therefore, rankings are used for the training instead of exact scores. To construct these pairs, we used the confidence score attributed by the students and the pm_score, a score based on the number of upvotes/downvotes for the Stack Exchange dataset. All pairs with equal scores were discarded.

C. Tutor model architecture and supervised training

GPT2 is a language model, trained on a large corpus of text from the web for text generation. Even though it is relatively small compared to more modern LLMs (only 352M), it still achieves impressive performance on a variety of benchmarks. However, it is not aligned on any particular task (Radford et al [18]). Here, we start from the "GPT2-medium" checkpoint available on Huggingface and fine-tuned it on our collected datasets without altering its architecture.

To fine-tune the model, we used the next token prediction framework as described in Radford et al. [6]. Using the collected text corpus $U = \{u_1, u_2, ..., u_k\}$, the objective is to maximize the log-likelihood of the sequence:

$$L(U) = \sum_i Log(P(u_i|u_1, ..., u_{i-1}))$$

The model learns to estimate the conditional probability P by minimizing the negative cross-entropy loss. This is done using the Adam optimization optimizer with a decreasing learning rate starting from 1e-4. This value was obtained by grid-search over a few possible values in the range $[10^{-4}, 10^{-5}]$. The batch size was 1 as a larger batch did not fit on the GPU used for training. All other hyperparameters of the GPT2 model were left to their default. The epoch number was set to 6. However, we used early stopping to mitigate any overfitting. If after 20'000 steps, about a third of an epoch, the training loss did not decrease more than 1e-4, we stopped the training and save the last model. In total, the training time was about 10 hours on an NVIDIA T4 GPU (16GB) on Google Cloud VM. Note that further hyperparameters tuning is necessary. Indeed, due to our budget constraint, we were unable to extensively tune the parameters, but it could lead to better performance.

Since the Stack Exchange dataset does not perfectly align with our desired behaviour. We first fine-tuned the model on the Stack Exchange data to teach it to answer questions related to science in general. Then, we further specialized it using the distil ChatGPT data.

D. Reinforcement learning with human feedback (RLHF) and Reward model

RLHF is a reinforcement learning (RL) technique using human feedback as a reward to guide training. To define the problem in the RL framework: The policy is the model taking an input sequence and returning a sequence of text. The observed state is the input The 50k dictionary is the action space. Finally, the reward function is a reward model trained on human preference ([4], [17])

Reward model architecture We choose a smaller-scaled model compared to the tutor model (86M vs 352M), the pre-trained DistilGPT2 model [19] as the base of the reward model. We preferred the use of a pre-trained model then to build the reward model from scratch. Indeed, a pre-trained model already has some capacity to encode efficiently the inputs. Moreover, we opted for this model as it has a larger maximum token length than BERT or ROBERTA for example. Lastly, due to our limited budget, we used the distilled version of GPT2 (82M vs 124M).

The DistilGPT2 model was originally designed and trained to be a text generation model. Here we want to use it to output a scalar reward reflecting human preference. Conveniently, an alternative architecture for sequence classification is available on Huggingface. Therefore, we loaded the pre-trained model using GPT2ForSequenceClassification. As we want our final model to output a scalar reward, we set the number of output neurons to one and pass the outputted logit through a linear layer to get the reward.

Reward model training For the training of the reward model, we used the same framework as described in Ouyang and al. [4]. Human-ranked pairs of demonstrations for the same questions are created using the confidence score of the collected data.

The loss is defined as the cross-entropy of the difference between the output rewards of the "good/winner" demonstration and the "bad/loser" one.

$$L = \frac{1}{N} \sum_i log(\sigma(r(x_i, y_{iw}) - r(x_i, y_{il})))$$

where N is the number of pairs in the batch, $r(x, y)$ is the scalar reward for the prompts x and the model's answer y, y_w and y_l represent respectively the best and worst generations according to human preference.

To avoid overfitting all pairs of demonstrations of the same questions are passed as a single batch element. Indeed, Ouyang et al. [4] found that shuffling the comparisons lead to overfitting after only one epoch. We used the following hyper-parameters for the training: learning rate = 1e-5, This value was obtained by grid-search over a few possible values in the range $[10^{-4}, 10^{-5}]$, Batch size = 1 (but as explained they are multi-batch), and number of epochs = 6. The model was evaluated on a validation set for every 500 iterations of the training loop. It was saved only when the achieved validation loss was better than the previous best one. Moreover, we found that training our model with the default parameters of the GPT2 configuration resulted in overfitting. Indeed, the training loss was much higher than the validation loss. Therefore, we decided to increase the regularization term of the model from 1e-5 to 5e-5. The total training time on NVIDIA T4 GPU (16GB) was about 4h on the Google Cloud platform.

The model was evaluated using the accuracy of the ranking of pairs of demonstrations from the validation set. The final model achieved an accuracy of 0.72. This is very satisfactory given that the task of ranking demonstrations is quite complex even for humans.

Fine-tuning using the PPO algorithm Next, to further improve the performance of our tutor, we decided to investigate the reinforcement learning framework. The first two ingredients of the algorithm are a model fine-tuned on demonstrations of the intended behaviour and a reward model that act as a guide to reinforce desirable behaviours and discourage unintended ones.

Briefly, during each iteration of the PPO algorithm, the model is prompted with a question unseen during the supervised fine-tuning step. It generates an answer based on its current policy. Next, the reward model evaluates the generation and provides feedback to guide the reinforcement learning process. To ensure the stability of the training, the PPO algorithm uses a clipped objective function. Avoiding any extreme policy change that may severely affect performance. [17] Following this iterative procedure, the model learns over time from its own interactions, gradually aligning its responses with the desired task (Fig. 3).

A common challenge in the training of models with RL is the ability of the model to exploit the reward model and to get an inflated reward but outputting gibberish. To avoid this a penalty term is introduced to the reward function:

$$R(x, y) = r(x, y) - KL(x, y)$$

where $r(x, y)$ is the reward computed by the RM and $KL(x, y)$ the KL-divergence between the current policy and the reference model. We maintain a reference model that remains unchanged throughout training, and we compare the generated responses of the new model to the reference using the KL divergence. In other words, the optimized model is penalized for straying too far away from the initial model. [4], [20]

Note that we described here the training procedure of RLHF, but due to limited computational resources, we decided against implementing it fully and executing it. Indeed, the procedure requires two models that need to fit onto one single GPU. Unfortunately, the GPU used for training does not have the necessary memory.

E. Decoding method and parameters

We conducted a series of experiments to determine the optimal parameters for our model's decoding method, specifically utilizing top-p nucleus decoding. The key advantage of top-p nucleus decoding is its ability to balance between randomness and coherence. As the threshold p is varied, the algorithm can produce outputs with varying degrees of diversity. For question-answering tasks, the randomness should be moderate to ensure the correctness and coherence of the outputs. But since we still want the mode to mimic human speech some randomness is necessary.

The parameters under consideration were *top_p*, *temperature*, and *max_new_tokens*. We explored a diverse set of combinations to identify which resulted in the most effective outcomes. Evaluation of the generated responses (model_answer) against the expected responses (gold_answer) was performed using a range of metrics, including ROUGE, BLEU, BERTScore, and our reward model. In addition, we also inspected the outputs to access the quality of the generations.

Preliminary results, based on two dozen generations, suggest that the optimal parameter settings are top_p = 0.70, temperature = 0.4, and max_new_tokens = 350 (input lengths are often 100 or under). These settings yielded superior performance on the distil ChatGPT dataset. For example, when increasing *top_p* to 80 and *temperature* to 1 or above was leading to very general generations with many repetitions (*eg "Let's think step by step" x4*), leading to less human-like text. To mitigate this issue, we set the no_repeat_ngram_size parameter to 3.

F. Baselines and evaluation metrics

The base model, GPT2, served here as a baseline for the evaluation of the final tutor model. We try to demonstrate here the effectiveness of the training procedure. This is done using a held-out test set from the training and the provided *prompts.json* file. To access quantitatively the quality of the generations we used the following metrics: ROUGE, BLEU, BERTscore, and our reward model. Moreover, we interacted with the model to better understand its behaviour.

IV. Results and analysis

Quantitative analysis: Utilizing the aforementioned generation parameters, we proceeded to apply them to the remaining portions of the Stack Exchange and ChatGPT datasets,

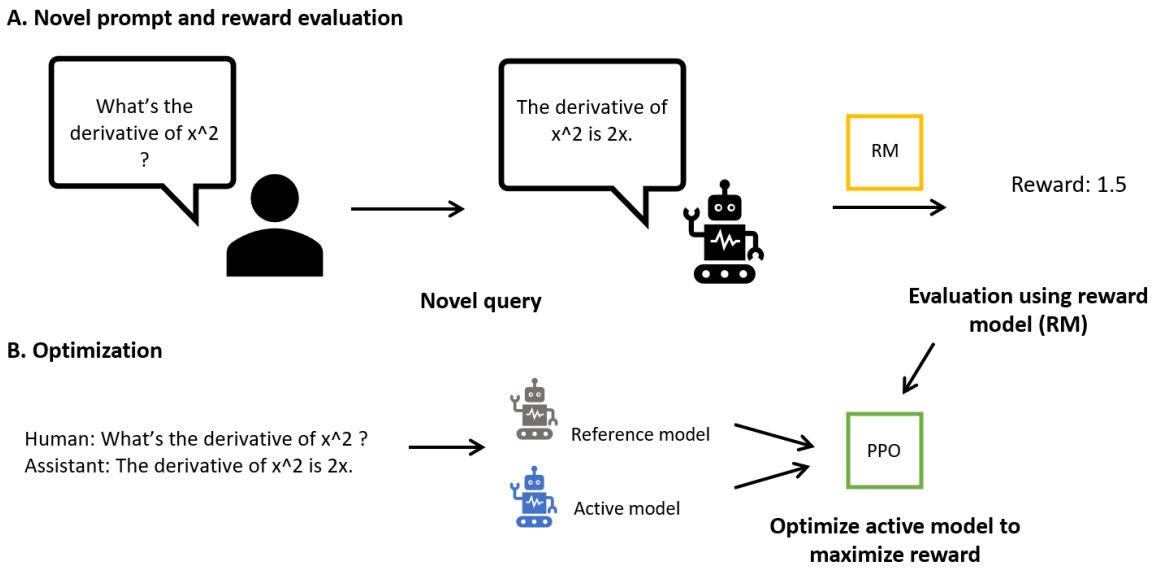

Fig. 3. Overview of the RLHF procedure; A) The supervised fine-tuned tutor model is prompted with questions unseen during its training. The generations are accessed by the reward model (RM) B) Using the estimated reward the tutor model is updated using the PPO algorithm to maximize the reward.

	Baseline ChatGPT	Tutor ChatGPT	Baseline Stack	Tutor Stack
ROUGE1	0.202	0.221	0.105	0.146
ROUGE2	0.046	0.049	0.015	0.019
BLEU	0.012	0.016	0.008	0.008
BERTscore F1	0.814	0.810	0.767	0.789
Model reward	3.285	3.385	3.015	3.199
Gold reward	3.650	3.650	2.603	2.603

Fig. 4. Validation results on the held-out test sets of the distilled ChatGPT (ChatGPT) and Stack Exchange (Stack) datasets on the base GPT2 model (baseline) and final tutor model (tutor). The best values per dataset are highlighted.

comprising 1192 and 470 entries respectively. The results of this application are illustrated in Fig. 4.

Note that the evaluation of the pre-trained model has been done only on 200 ChatGPT prompts and 100 Stack prompts due to time and GPU constraints. We see encouraging results: our finetuned model outperforms the baseline according to almost all metrics, with a difference of +0.1 on ChatGPT and +0.2 on Stack according to our reward model grades. Here are what we can deduce from the different metrics :

- ROUGE: A marginal overlap of unigrams is observed, while instances of bigram overlaps are notably infrequent. This phenomenon could partially be attributed to the prevalence of mathematical and chemical expressions in many questions. Such expressions pose a significant challenge for generative language models to correctly replicate.
- BLEU: Given that this metric also employs n-grams along with a brevity penalty, the same conclusions are drawn.
- BERTscore: The evaluation of contextual embeddings through cosine similarity yielded a value around 0.80, a decent score that signifies a reasonable alignment of the generated and reference sequences. This score seems to indicate no significant improvement between the baseline and the final model.
- Reward Model: Interestingly, the reward increased for the final tutor model. Meaning that the answer seems to be more aligned with the behaviour considered by the reward model as good.

Qualitative analysis: Next, we qualitatively analyzed the generated responses. A random sample of fifteen instances was taken from each dataset, and the generated responses were evaluated on the basis of solution accuracy, conciseness, coherence, and the overall syntactic structure of the sequences.

We observed several peculiarities during the analysis of the generated responses. The model is producing outputs in the first person narrative (*"I've found that"*, *"My solution is"*), a trait that was most likely learned from the Stack Exchange data. Note that the data from Stack Exchange, although filtered extensively, may not have been the ideal training set for our model. The dataset contained instances that referenced external demonstrations or provided only partial solutions to the questions as the exchange is sometimes long discussions, which could have potentially affected the model's performance.

A surprising observation we made was finding prompts from the initial step (where we obtained and graded individual answers) appearing in our generated outputs. Phrases like *"Please give me a confidence score between 1 and 5 (1 meaning you are very confident in your answer) and a short explanation of the reasoning behind your answer. Let's think step-by-step."* or *"Please answer the following question with one correct choice and a simple explanation of the choice."* surfaced in the generated responses. This phenomenon could be attributed to the fact that sometimes the inputs were brief, causing our next-token generation model to attempt to "complete the question." It appears that the model has learned to wrap up the question before it proceeds to provide an answer.

Moreover, the model did not provide accurate solutions in calculus, a known limitation of many Language Models (LMs), hence not surprising. Also, the model demonstrated its capability to construct reasoning, for example, "*First, we'll look at the mRNA expression data.*" We also note the poor performance when the model is prompted in French. It sometimes answers in French, but the sentences are nonsensical. This was expected as all French interactions were translated into English.

Overall, the syntactic structure of the generated responses was satisfactory. However, often instead of answering the question directly, the model seems to have learned to mimic the prompting strategies found in the training dataset. But, it still often shows the intended behaviour of explaining its reasoning to respond to the question after some rambling. However, it often lacks coherence and provide incorrect solutions.

Limitations and future improvements This highlights areas for potential improvements and optimizations in future iterations of our model. Indeed, one should get rid of any additional prompting after the question to avoid the unexpected behaviour of completing the question. Moreover, more data is needed to make sure that the model is generalizable and does not overfit. As the topics that the model is supposed to answer are relatively complex, we should increase the model size. For instance, one could imagine using the same training procedure with more data on a base model such as LLaMA. Indeed, it has been shown that starting from this model, one could get a similar performance as ChatGPT. [8], [9], [10]. Lastly, the model was not fine-tuned using RLHF. Therefore, after obtaining a stronger base model through supervised learning, we should implement it. However, all these points require significantly more computational budget and time.

V. Conclusion

In this study, we have demonstrated the potential of utilizing a fine-tuned language model, specifically GPT-2 medium, as a tutoring tool. Through our experimental design, we have exhibited how reinforcement learning with human feedback (RLHF) and a reward model can be instrumental in optimizing the performance of such a model.

Even though GPT-2's capacity is relatively small compared to more recent large language models, its performance in the context of our research was promising. However, it is important to note that the alignment with the specific task of tutoring in the context of EPFL courses was not perfect, likely due to the lack of sufficient data and artefacts of the prompting engineering in the training set.

This investigation demonstrates the feasibility of using generative language models for tutoring purposes. It also highlights the need for further research in this domain to realize the full potential of such an approach. Accumulating more and more specialized training data is a critical step for refining the performance of the tutor model. In parallel, it would be beneficial to employ larger-scale models to leverage their enhanced capacity for understanding and generating complex text sequences.

The implementation of reinforcement learning with human feedback procedure is another exciting possibility that may significantly improve the model performance by iteratively refining the policy based on feedback. In conclusion, this research provides a first step towards the goal of creating an effective tutor whispering to the ears of EPFL students.

VI. Contributions

All: Conceptualization of the training procedure; A.D.: Collection of the additional ChatGPT demonstrations and questions.; P.H.: Data pre-processing for tutor and reward model training; P.B. Implementation of the training loop for the reward models; A.D. Implementation of the training loop for tutor model; A.D. and P.B.: Training of both models and tuning hyperparameters. P.H. Tuning of the decoding algorithm; P.B. and P.H.: Evaluation and analysis of the model's behaviour All: Write the report.
Note: A.D = Alexia Dormann; P.H. = Pierre Hellich; P.B. = Paul Boulenger

References

[1] Chen Ling, Xujiang Zhao, Jiaying Lu, Chengyuan Deng, Can Zheng, Junxiang Wang, Tanmoy Chowdhury, Yun Li, Hejie Cui, Xuchao Zhang, Tianjiao Zhao, Amit Panalkar, Wei Cheng, Haoyu Wang, Yanchi Liu, Zhengzhang Chen, Haifeng Chen, Chris White, Quanquan Gu, Carl Yang, and Liang Zhao. Beyond one-model-fits-all: A survey of domain specialization for large language models, 2023.

[2] Vagelis Plevris, George Papazafeiropoulos, and Alejandro Jiménez Rios. Chatbots put to the test in math and logic problems: A preliminary comparison and assessment of chatgpt-3.5, chatgpt-4, and google bard, 2023.

[3] Haoye Tian, Weiqi Lu, Tsz On Li, Xunzhu Tang, Shing-Chi Cheung, Jacques Klein, and Tegawendé F. Bissyandé. Is chatgpt the ultimate programming assistant – how far is it?, 2023.

[4] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.

[5] Nathan Lambert, Lewis Tunstall, Nazneen Rajani, and Tristan Thrush. Huggingface h4 stack exchange preference dataset, 2023.

[6] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.

[7] Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences, 2023.

[8] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

[9] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023.

[10] Xinyang Geng, Arnav Gudibande, Hao Liu, Eric Wallace, Pieter Abbeel, Sergey Levine, and Dawn Song. Koala: A dialogue model for academic research. Blog post, April 2023.

[11] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.

[12] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions, 2023.

[13] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.

[14] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners, 2023.
[15] Zekun Li, Baolin Peng, Pengcheng He, Michel Galley, Jianfeng Gao, and Xifeng Yan. Guiding large language models via directional stimulus prompting, 2023.
[16] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020.
[17] Nathan Lambert, Louis Castricato, Leandro von Werra, and Alex Havrilla. Illustrating reinforcement learning from human feedback (rlhf). *Hugging Face Blog*, 2022. https://huggingface.co/blog/rlhf.
[18] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
[19] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. In *NeurIPS EMC*$^2 Workshop$, 2019.
[20] Edward Beeching, Younes Belkada, Kashif Rasul, Lewis Tunstall, Leandro von Werra, Nazneen Rajani, and Nathan Lambert. Stackllama: An rl fine-tuned llama model for stack exchange question and answering, 2023.